

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт к практическому заданию №2
по дисциплине «Системное программное обеспечение 2»

Студент: Горинов Даниил Андреевич
Группа: Р4116

Санкт-Петербург
2025/2026 учебный год

1 Цели

Изучить механизмы синхронизации, реализовать объект последовательного потока данных и показать корректность его работы на тестовой программе по варианту.

Для варианта 4 используются SQL-выборки лабораторной работы №3, режим потоков — синхронный неблокирующий. Выборки представлены как map-reduce pipeline из независимых обработчиков, связанных последовательными byte streams.

2 Задачи

1. Изучить внутреннюю логику синхронных потоков и пассивного ожидания.
2. Модифицировать планировщик из SP06 для поддержки ожидания готовности группы потоковых событий.
3. Описать SQL-выборки варианта и проверить их на базе PostgreSQL `ucheb`.
4. Разложить каждую выборку на этапы `source`, `filter`, `join`, `group/aggregate` и `formatter`.
5. Реализовать VM-демонстрацию конкурентного выполнения обработчиков.
6. Сравнить результаты VM-демонстрации с количеством строк, полученным SQL-запросами.

3 Описание работы

Точные SQL-запросы находятся в `sql/variant_lab3.sql`. Проверка выполнялась на базе `ucheb`; снимок результата сохранён в `sql/validation_snapshot.md`.

VM-демонстрация находится в `tests/sql_pipeline_demo.asm`. В ней каждая выборка представлена набором обработчиков. Обработчики получают элементы через входные byte streams и пишут результат в выходные byte streams. Неблокирующая операция чтения или записи при отсутствии готового события возвращает состояние `would-block`; после этого обработчик переводится в `GROUP_WAIT`, а планировщик отдаёт квант другому обработчику.

Запуск:

```
make -C SP07 remote-demo
```

Команда запускает `Portable.RemoteTasks.Manager.exe` через `ExecuteBinaryWithIo`, чтобы работали `SimpleClock` и `SimplePic`, затем проверяет `results/sql_pipeline_demo.stdout.t` скриптом `tools/check_sql_pipeline_output.py`.

Основные файлы:

- `tests/sql_pipeline_demo.asm` — модель SQL map-reduce pipeline;
- `sql/variant_lab3.sql` — запросы варианта;
- `sql/validation_snapshot.md` — проверенные количества строк;
- `spo7.target.pdsl` и `devices.xml` — целевая VM и устройства таймера;
- `Validation.md` — команды проверки.

4 Аспекты реализации

Синхронный stream не хранит большую очередь данных: передача элемента считается выполненной, когда producer и consumer согласованы. В неблокирующем режиме потоковая операция не занимает CPU в ожидании. Если вход пуст или выходной stream не готов, обработчик фиксирует GROUP_WAIT, сохраняет своё состояние и возвращает управление scheduler runtime.

Планировщик использует схему из SPO6: SimpleClock формирует timer IRQ, SimplePic передаёт управление обработчику, а обработчик выполняет один шаг текущего processor stage. За счёт этого разные этапы одной выборки продвигаются независимо, а ожидание готовности stream-событий остаётся пассивным.

Декомпозиция выборок построена в терминах map-reduce:

- parser/source читает таблицу;
- filter пропускает записи по условию;
- join соединяет два входных потока по ключу;
- group/aggregate считает COUNT, AVG, HAVING и устраняет дубликаты через группировку;
- formatter пишет итоговый поток результата.

Количество обработчиков и ожидаемых GROUP_WAIT задаётся таблицами в VM-программе. Поэтому степень параллелизма меняется данными сценария, а не переписыванием логики планировщика.

5 Результаты

Последний зафиксированный RemoteTasks-прогон:

```
assemble = d0526229-a670-42b0-ae59-19c34a10bea0
run = 0937d3a3-2dfd-4e25-b476-12d35df3b053
```

Результат VM-демонстрации:

```
SP07 SQLMR
MODE SYNC-NB GROUP-WAIT
Q1 R=1 P=5 W=1
Q2 R=0 P=5 W=1
Q3 R=5004 P=4 W=1
Q4 R=40 P=6 W=2
Q5 R=85 P=7 W=2
Q6 R=0 P=8 W=2
Q7 R=0 P=5 W=1
IRQ=48 GW=10
OK
```

Обозначения: R — число строк результата SQL-запроса в PostgreSQL, P — число обработчиков в pipeline, W — число событий группового ожидания. Суммарно выполнено 48 timer-driven шагов и 10 passive group waits.

Запрос	Смысл	Строк
Q1	RIGHT JOIN типов ведомостей и ведомостей	1
Q2	INNER JOIN людей, обучений и учеников	0
Q3	число пар фамилия-имя после группировки	5004
Q4	планы с двумя группами на кафедре ВТ	40
Q5	студенты группы 4100 со средней оценкой выше средней группы 1101	85
Q6	студенты первого курса заочной формы до 2012-09-01	0
Q7	число отличников группы 3100	0

6 Выводы

Последовательные byte streams и групповое пассивное ожидание позволяют представить SQL-выборки как набор конкурентных обработчиков. VM-демонстрация воспроизводит количества строк, полученные PostgreSQL-запросами, и показывает, что при **would-block** обработчик не ждёт активно, а уступает квант другим этапам pipeline. Это соответствует требуемому синхронному неблокирующему режиму варианта.